# Rectangular Batching for Disjoint Shortest Paths on DAGs

ZIJIAN ZENG, UCSI University, Malaysia

Given a positive-weight directed acyclic graph and prescribed source–target pairs, the disjoint shortest paths problem asks for mutually vertex-disjoint paths, each shortest for its own pair in the original graph. For every fixed integer $k \geq 3$, we give a deterministic detection algorithm with running time $O_k(m + n + n^{k+0.3})$. Original-weight preprocessing uses $O_k(m + n)$ additions and comparisons; the remaining bound includes elementary arithmetic on the new $O_k(\log(n + 2))$-bit operands. Thus, in the usual $m = O(n^2)$ input convention, $k = 3$ and $\delta = 0.7$ answer the question of Akmal, Vassilevska Williams and Wein asking for an $O(n^{k+1-\delta})$ algorithm for some fixed $k \geq 3$.

The algorithm accelerates the established minimum-coordinate frontier recurrence. At each chronological split, a disjoint partition by the target minimum permits all transitions to be grouped into rectangular Boolean products. We charge repeated packing, scattering and products over the full recursion. A fixed rational tensor identity, obtained from an established rectangular multiplication bound, can be specialized modulo a sufficiently large prime to give exact Boolean products with bounded operands. An explicit Strassen-based alternative gives $O_k(m + n + n^{k-2+\log_2 7})$ word operations. The method also applies to prescribed disjoint paths with label-specific edge sets sharing a common acyclic order.

Author's Contact Information: Zijian Zeng, UCSI University, 1002266693@ucsiuniversity.edu.my.

## 1 The result and its context

In $k$-disjoint shortest paths ($k$-DSP), the input is a graph together with $k$ ordered terminal pairs $(s_i, t_i)$. The paths must respect the prescribed pairing, avoid the prohibited vertex intersections, and each have the globally shortest length for its own pair in the original graph. Minimizing a sum of path lengths subject to disjointness is a different task: it may allow one path to become longer to accommodate another. Here shortestness is fixed before any conflicts are resolved.

For DAGs, the established bound is $O(mn^{k-1})$. Akmal, Vassilevska Williams and Wein state this bound and ask whether any fixed $k \geq 3$ permits a polynomial saving over the dense $n^{k+1}$ exponent, or whether a standard hypothesis rules out such a saving [1], full version, Open Problem 3. Their two-pair linear-time detection result does not answer that question. We give a positive answer for every fixed $k \geq 3$.

Theorem 1.1. *For every fixed integer $k \geq 3$, prescribed vertex-disjoint shortest-path detection on a positive-weight DAG with $n$ vertices and $m$ edges is deterministic in*

$$O_k(m + n + n^{k+0.3})$$

*time in the original-weight addition/comparison model. New algebraic operands occupy $O_k(\log(n + 2))$ bits. Their elementary arithmetic cost is included in the displayed bound. This includes strict vertex disjointness, endpoint-only sharing, and the convention forbidding only internal/internal intersections, with the relevant convention fixed in advance.*

For ordinary graphs $m = O(n^2)$, the case $k = 3$ is $O(n^{3.3})$, so $\delta = 7/10$ resolves the full existential question. If arbitrary parallel arcs are listed, the $O(m)$ input scan remains explicit. No bounded-width, unit-weight, unique-shortest-path, sparse-instance or distances-given assumption is made. For $B$-bit integer input weights, actual distance preprocessing costs must be added, for example $O_k((m + n)(B + \log(n + 2)))$ bit operations using elementary additions and comparisons. Arbitrary input weights are not assumed to fit a word.

*What changes in the known algorithm.* The frontier table has $O_k(n^k)$ entries, but its explicit transitions can cost $\Theta(n^{k+1})$. We retain that table and the known rule that only its least coordinate moves. We accelerate the transitions. At a chronological split with halves $L, R$, every updated target has minimum in $R$. Its moved coordinate either lies in $R$, or lies later and a context coordinate lies in $R$. Selecting the first such context coordinate gives exactly $k$ disjoint output classes. Each class becomes a contraction over $L$, with an entire labelled context kept together in one matrix column. This preserves path compatibility and makes the total product dimensions small enough. Charging all dyadic nodes and all repeated data movement is essential. OpenAI Codex materially assisted the batching construction, proof development and diagnostic implementation; its full roles are described at the end.

*Prior tools and precise distinction.* Ordered tuple graphs and terminal devices precede this work. Eppstein's report [4], Sections 2–4, gives an $O(mn^{d-1} + K)$ enumeration bound for $K$ feasible $d$-tuples. The minimum-coordinate recurrence is displayed explicitly in Fleischer, Ge, Li and Zhu [5], Section 2.1, equations (1)–(2), where it is attributed to Perl–Shiloach. Bérczi and Kobayashi [2], Proposition 10, use label-specific permitted edges and a tuple graph for shortest paths. The present proof rederives those semantics to specify exactly what is batched; it does not claim the recurrence or shortest-path filtering as new. Chronological closure acceleration using Boolean multiplication is already central in Valiant [8]. The contribution here is the particular output partition, rectangular context packing, and full global charge for the labelled frontier problem. Applying generic closure to all $n^k$ states does not itself establish the claimed bound.

The numerical consequence uses Le Gall and Urrutia's aspect-two exponent bound [6], Table 3 and Section 2.2. Later bounds of Vassilevska Williams, Xu, Xu and Zhou [9], Table 1, are stronger; no current

matrix multiplication record is claimed here. Choudhary, Kumar and Saggi [3], Theorem 1.1, instead treat two prescribed pairs in more general directed graphs. Their $O(mn \log n)$ detection result addresses a different number of pairs and does not give this fixed-$k \geq 3$ DAG bound.

## 2 Reduction to labelled paths in a common order

Compute distances $d_i$ from each original source by topological relaxation in the original DAG. If $t_i$ is unreachable, return NO. For label $i$ retain exactly the edges $(u, v)$ with finite $d_i(u)$ and

$$d_i(v) = d_i(u) + w(u, v). \tag{1}$$

Every shortest $s_i$–$t_i$ path uses only these edges, since each of its prefixes is shortest. Conversely, summing (1) along any retained $s_i$–$t_i$ path telescopes to $d_i(t_i)$. Thus its original length is shortest. This costs $O_k(m+n)$ original-weight operations. All retained graphs share the original topological order; distances are never recomputed after conflicts or deletions.

First use strict disjointness, including endpoints. Endpoints belonging to different labels cannot coincide; if they do, return NO. Within one pair, $s_i = t_i$ is allowed and represents the trivial path. Append a private sink $z_i$ for each label, after all original vertices, with the sole label-$i$ completion edge $t_i \to z_i$. Appending or removing these edges preserves feasibility. A path passing through $t_i$ without completing cannot later return to it in a DAG. All unfinished positions precede all completed sinks.

For the convention allowing endpoint/endpoint sharing only, replace each terminal vertex $v$ by a private copy $v_i$ for every label having $v$ as an endpoint. A label without that endpoint cannot visit $v$. Map its retained edges to its appropriate private copies, forbid entry into its source and exit from its target except for completion, and leave all other vertices shared. Endpoint uses separate, while every forbidden internal intersection remains a collision or a prohibited visit. If only internal/internal intersections are forbidden, keep in addition one shared internal copy of each terminal $v$, available precisely to labels for which $v$ is not an endpoint. Two internal uses then collide; endpoint/internal uses do not. A trivial pair uses its single endpoint copy and completes.

These transformations are equivalent in both directions by mapping each use of an original vertex to its designated copy. Only $O_k(1)$ vertices are added, since there are at most $2k$ terminal vertices and $k$ is fixed. Copies of a vertex occupy consecutive positions in the common order. An original edge always joins different original ranks, so every mapped edge is forward. We have reduced all stated conventions to strict disjointness in $k$ label-specific forward-edge graphs. Pad this order with isolated vertices to a power of two $N = O_k(n)$, indexed by $0, \ldots, N-1$.

## 3 The frontier recurrence and chronological invariant

A valid state is an ordered tuple $q = (q_1, \ldots, q_k)$ of pairwise distinct positions, with $\rho(q) = \min_i q_i$. Put $F(q) = 0$ initially, except for the source tuple, where $F = 1$. In the frontier graph only the label $i$ occupying the unique minimum $u = q_i$ can move, along an allowed edge $(u, x)$, provided the resulting tuple remains valid. Since $x > u$, the minimum strictly increases.

Lemma 3.1. *The private-sink tuple is reachable in the frontier graph if and only if there is a feasible family of prescribed labelled disjoint paths.*

Proof. Append each frontier move to the corresponding path prefix. Every abandoned vertex is smaller than every current coordinate. Initially there are none; after moving the least coordinate, the newly abandoned vertex is smaller than its replacement and all unchanged coordinates. Thus a later destination cannot hit an abandoned vertex. The validity check forbids a hit on any current coordinate, proving soundness.

For completeness fix a feasible family ending at the private sinks. Move the least token along its path in that family. If any token is unfinished, an unfinished token is least, because every original vertex precedes every private sink. Its next edge exists, and disjointness makes the move legal. The total number of remaining edges decreases, so the sink tuple is reached. □

Let $A_i$ be label $i$'s Boolean adjacency matrix. For a valid target $p$, every incoming transition moving label $i$ has exactly the form

$$F\big(p[i:=u]\big) \ \wedge \ A_i(u, p_i), \qquad u < \rho(p). \tag{2}$$

Here $p[i:=u]$ replaces coordinate $i$ by $u$; invalid states always have value zero. The target value is the OR of these terms and its initial seed. Each term uses one complete predecessor tuple, so it never merges witnesses chosen independently for different paths. At intermediate tuples, $F$ means frontier-graph reachability; no stronger prefix interpretation is needed.

Recursively finalize states in increasing order of their minimum. At a node $I = [l, r]$, split into equal halves $L = [l, h]$ and $R = [h + 1, r]$ of size $b$. Finalize $L$, apply all transitions with source minimum in $L$ and target minimum in $R$, then finalize $R$. A singleton needs no internal transition. At entry to a call, transitions from minima below $l$ have already been applied, and all smaller-minimum states are final. Every transition with source minimum $u$ and target minimum $v > u$ belongs to the unique node whose split separates $u$ into $L$ and $v$ into $R$. Induction proves that each transition is applied once, with final source values, before its target is finalized. A seed later in the order remains present throughout this process.

## 4 The output partition and rectangular batching

At a cross step define $S = [h + 1, N - 1]$ and $T = [r + 1, N - 1]$, so $S = R \cup T$. Fix the moved label $i$ and order the other $k - 1$ labels. Write their positions as one labelled context $c = (c_1, \ldots, c_{k-1})$. For $u \in L$ let $D(u, c) = F(c \text{ with } u \text{ inserted at label } i)$. Only contexts in $S^{k-1}$ are considered. Invalid contexts give zero entries. If $x$ is the new position of label $i$, partition the outputs as follows:

$$C_0 : \quad x \in R, \quad c \in S^{k-1}; \tag{3}$$

$$C_j : \quad x \in T, \quad c_j \in R, \quad c_a \in T \ (a < j), \quad c_a \in S \ (a > j), \quad 1 \le j \le k - 1. \tag{4}$$

These classes are disjoint and cover exactly the outputs with minimum in $R$. If $x \in R$ use $C_0$. Otherwise $x \in T$ and the first context coordinate in $R$ uniquely determines $j$. Targets with repeated coordinates will be masked out. In particular the moved coordinate may lie beyond the current interval $I$; excluding such destinations would be wrong. For $k = 3$ the context domains are respectively $S \times S$, $R \times S$ and $T \times R$, with $x \in R, T, T$ respectively.

For one class, batch at most $b$ destinations and at most $b^{k-1}$ contexts. With zero padding, multiply the Boolean matrices

$$P[x, u] = A_i(u, x) \quad (b \times b), \qquad Q[u, c] = D(u, c) \quad (b \times b^{k-1}). \tag{5}$$

For every output $(x, c)$, the product is the OR over $u \in L$ of exactly the corresponding terms in (2). Scan the result, discard invalid target tuples, and OR it into $F$. Inputs have minimum in $L$; all writes have minimum in $R$. Therefore no product in this cross step depends on another product's output. Label, class and batch order do not affect the result.

Theorem 4.1. *For fixed $k \geq 2$, let $\mathcal{B}_k(b)$ be the cost of deterministic exact Boolean multiplication of a $b \times b$ matrix by a $b \times b^{k-1}$ matrix, including input and output arrays. The labelled common-order disjoint-path problem is solvable in*

$$O_k\left(N^k \log(N+2) + \sum_{b \in \{1,2,4,\ldots,N/2\}} (N/b)^k \mathcal{B}_k(b)\right). \tag{6}$$

*Its space is $O_k(N^k)$ words plus the multiplication primitive's workspace.*

Proof. Class 0 has at most $N^{k-1}$ contexts and one destination batch, hence at most $(N/b)^{k-1}$ products. Class $j > 0$ has at most $bN^{k-2}$ contexts and at most $N/b$ destination batches. Its number of context batches is at most $(N/b)^{k-2}$: this is an integer after power-of-two padding, so rounding up cannot increase the upper bound. Each class therefore uses at most $(N/b)^{k-1}$ products. Empty domains cause no calls. Across $k$ labels and $k$ classes the bound is $k^2(N/b)^{k-1}$ products per node.

Every product packs $b^2 + b^k$ entries and scans, masks and scatters $b^k$ entries. Charge $O_k(b^k)$ every time, including when repacking an operand for a different destination batch. Context enumeration costs $O_k(N^{k-1})$ per node and fits within $O_k((N/b)^{k-1}b^k) = O_k(N^{k-1}b)$. Contexts can be enumerated in streaming batches; there is no full $N^k$ table scan at any one node. Fixed $k$ gives constant-time tuple indexing on $O_k(\log(N+2))$-bit words.

There are $N/(2b)$ nodes at half-width $b$. Their multiplication costs sum to $O_k((N/b)^k \mathcal{B}_k(b))$, and their other data movement costs sum to $O_k(N^k)$. Add the $O_k(N^k)$ initialization and sum all levels to obtain (6). The table, adjacency matrices, streamed packed arrays and recursion stack take $O_k(N^k)$ words; any additional primitive workspace must be added. Correctness follows from the chronological invariant, (5), and Lemma 3.1. □

In particular, if $\mathcal{B}_k(b) = O(b^\tau)$ with fixed $\tau > k$, the geometric sum in (6) is $O_k(N^\tau)$ and absorbs $N^k \log(N+2)$. This strict inequality will hold for both implementations below. Additional global bit overhead is charged explicitly.

## 5 An explicit square-multiplication implementation

Partition the $b^{k-1}$ context columns into $b^{k-2}$ blocks of $b$ columns. Each block is an integer product of two Boolean square matrices, followed by testing positivity. Strassen's seven-product identities [7] give

$$\mathcal{B}_k(b) = O_k(b^{k-2+\alpha}), \qquad \alpha = \log_2 7.$$

This includes the $b^k$ array entries since $\alpha > 2$. The exponent lies strictly between $k$ and $k+1$. Theorem 4.1 gives $O_k(N^{k-2+\alpha})$ word operations, with $O_k(N^k)$ live words when square products are streamed. This already supplies a polynomial dense saving.

For completeness, exact arithmetic uses bounded operands. At Strassen depth $d$, input sums have magnitude at most $2^d$. Leaf products have magnitude at most $b^2$, and each return level combines at most four signed terms per entry. Thus all intermediate integers have magnitude $O(b^4)$ and occupy $O(\log(N+2))$ bits. Positivity, rather than parity, gives the Boolean answer. Elementary multiplication introduces at most a $O(\log^2(N+2))$ bit factor; this factor is separate from the displayed word-operation bound. The next implementation proves Theorem 1.1 including that elementary bit cost.

## 6 An exact bounded-operand rectangular primitive

We explain the algebraic dependency because a field-operation exponent alone does not prove the claimed Boolean time. Write

$$\langle a, b, c\rangle = \sum_{\mu=1}^{a}\sum_{\nu=1}^{b}\sum_{\xi=1}^{c} X_{\mu\nu}Y_{\nu\xi}Z_{\xi\mu}$$

for the matrix multiplication tensor. Le Gall and Urrutia [6], Section 2.2 and Table 3, establish an exponent at most 3.251640 for $\langle q, q^2, q\rangle$. Their polynomial-degeneration formalism is over a field; specialize it to $\mathbb{Q}$. Choosing an exponent strictly between that bound and 3.26, and then a sufficiently large fixed $q_0$, absorbs the asymptotic constant and gives a rational border-rank decomposition with $r_0 < q_0^{3.26}$ scalar products. No infimum is assumed to be attained.

Lemma 6.1. *There exist a fixed integer $q$ and one exact rational bilinear algorithm of rank $r < q^{3.27}$ multiplying a $q \times q$ matrix by a $q \times q^2$ matrix.*

Proof. Fix the preceding polynomial degeneration. Its leading nonzero coefficient is $\lambda^s\langle q_0, q_0^2, q_0\rangle$ and its degree is at most $D$, for fixed $s, D$. Its $t$-fold tensor power has leading coefficient at degree $st$ equal to $\langle q_0^t, q_0^{2t}, q_0^t\rangle$ and degree at most $Dt$. Interpolate this coefficient from $Dt + 1$ distinct rational evaluations. Each evaluation has rank at most $r_0^t$, so exact rational rank is at most $(Dt + 1)r_0^t$. For a sufficiently large fixed $t$, this is less than $(q_0^t)^{3.27}$. Put $q = q_0^t$.

Tensor rank is invariant under permuting its three variable groups. To make the required orientation explicit, for indices $a, c \in [q]$, $b \in [q^2]$, set

$$X'_{ac} = Z_{ca}, \qquad Y'_{cb} = Y_{bc}, \qquad Z'_{ba} = X_{ab}.$$

Then $\sum_{a,b,c} X_{ab}Y_{bc}Z_{ca} = \sum_{a,c,b} X'_{ac}Y'_{cb}Z'_{ba}$, which is $\langle q, q, q^2\rangle$. This exchanges the first and third variable groups and reindexes them. Reading the coefficients of the third group gives the required bilinear algorithm with the same exact rational rank. □

All coefficients of this one base identity are fixed rational constants. Let $H$ be the product of their nonzero denominators, and choose a fixed integer $C > |H| + 2$. For the $N$-vertex instance choose a prime $p \in (K, 2K)$ with $K = \max(N, C)$, using a deterministic sieve and Bertrand's theorem. No denominator vanishes modulo $p$, so reducing the base identity modulo $p$ preserves it exactly. One prime is reused throughout the instance. Its construction costs at most $O(K \log^2(K + 2))$ elementary bit operations, which is within the final bound, with constants depending on the fixed base. Reducing the fixed coefficients is also a fixed setup cost.

To multiply a $b \times b$ matrix by a $b \times b^2$ matrix, pad to $M = q^a$ with $b \le M < qb$ and recurse using the fixed base over $\mathbb{F}_p$. If necessary add zero products to arrange $r > q^3$ while retaining $r < q^{3.27}$; the fixed base can be enlarged in advance. All dense linear maps and output combinations then satisfy

$$T(M) \le rT(M/q) + O_q(M^3), \qquad T(M) = O_q(M^{\log_q r}) = O_q(b^{3.27}).$$

The additive term includes every input linear combination and all outputs; padding costs $O_q(b^3)$ and is included. Residues use $O(\log(N + 2))$ bits, with elementary modular multiplication and reduction costing $O(\log^2(N + 2))$ bit operations. The original Boolean factors have integer output entries in $[0, b]$, even after zero padding. Since $p > N \ge b$, a residue is nonzero exactly when its original integer entry is positive. Hence testing nonzero computes the Boolean product exactly, without parity cancellation or random error. We have proved, for this global modulus,

$$\mathcal{B}_3(b) = O(b^{3.27} \log^2(N + 2)). \tag{7}$$

The dependence on $N$ in this implementation is solely the global operand bit bound; a standalone primitive could choose its own prime above $b$.

For $k > 3$, divide the $b^{k-1}$ columns into $b^{k-3}$ blocks of $b^2$ columns and use (7). The resulting cost is

$$O_k(b^{k+0.27}\log^2(N+2)),$$

including the $b^k$ array entries. Substitution in (6) yields

$$O_k\big(N^k\log(N+2) + N^{k+0.27}\log^2(N+2)\big) = O_k(N^{k+0.3}).$$

The prime setup is smaller. Combining this with Section 2 and $N = O_k(n)$ proves Theorem 1.1.

The rational base may be extremely large. It is one finite identity, hardwired independently of graph size, so the construction defines a uniform algorithm rather than size-dependent advice. No coefficient list or fast rectangular implementation is supplied here. The exponent depends on the cited degeneration theorem and the exact conversion just proved. The explicit algorithm of Section 5 has no such implementation dependency. Interpolation, tensor permutation and prime specialization are established algebraic principles; no improvement to the underlying matrix multiplication exponent is asserted.

## 7 Verification, scope and limitations

The proof applies to any fixed collection of label-specific forward-edge graphs on a common order, not only tight-edge graphs. In particular it also applies to ordinary prescribed disjoint paths on DAGs. It preserves all labels and detects feasibility; no search-to-decision or reporting bound is asserted. The $N^k$ frontier table remains, and dense-instance speedup does not imply an improvement over every sparse-instance baseline. No extension to cyclic digraphs, optimal exponent or matching lower bound is proved.

Exact finite checks accompany the argument. The research implementation compared 145 complete frontier reached sets on arbitrary labelled and weighted instances, including $k = 2, 3, 4, 5$. A separately written implementation compared 4,718 reached sets: all triples of labelled forward-edge graphs on three vertices with all ordered distinct seeds (3,072 cases), all common-edge DAGs on four vertices with ordered three-token seeds (1,536 cases), and 110 further cases. It also compared 120 weighted original instances under three endpoint conventions, giving 360 original shortest-path comparisons; cases included 181-bit weights, ties, unreachable targets, repeated terminals and trivial pairs. All comparisons matched. These finite tests check indexing and semantics. They do not establish the asymptotic bound, implement the fast tensor primitive, or replace any proof. The executable checks and their precise finite coverage are retained with the research artifacts.

*AI methodology.* OpenAI Codex materially assisted the research program, including the batching construction, proof development, counterexample searches, verification code, primary-literature comparison, and drafting of all sections. Separate Codex contexts performed an internal mathematical review and wrote an independent finite verifier. These model-assisted checks are not external peer review or a formal proof certificate. The manuscript's mathematical justification is the argument and identified dependencies above; no claim of independent human verification is made here. The listed human author retains responsibility for any use or dissemination of this work.

## References


[1] Shyan Akmal, Virginia Vassilevska Williams, and Nicole Wein. 2024. Detecting Disjoint Shortest Paths in Linear Time and More. In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024) (Leibniz International Proceedings in Informatics, Vol. 297)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:17. doi:10.4230/LIPIcs.ICALP.2024.9 Full version: arXiv:2404.15916v2, especially Open Problem 3.

[2] Kristóf Bérczi and Yusuke Kobayashi. 2017. The Directed Disjoint Shortest Paths Problem. In *25th Annual European Symposium on Algorithms (ESA 2017) (Leibniz International Proceedings in Informatics, Vol. 87)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 13:1–13:13. doi:10.4230/LIPIcs.ESA.2017.13

[3] Keerti Choudhary, Amit Kumar, and Lakshay Saggi. 2026. Efficient Algorithms for the Disjoint Shortest Paths Problem and Its Extensions. In *17th Innovations in Theoretical Computer Science Conference (ITCS 2026) (Leibniz International Proceedings in Informatics, Vol. 362)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 39:1–39:23. doi:10.4230/LIPIcs.ITCS.2026.39

[4] David Eppstein. 1995. *Finding Common Ancestors and Disjoint Paths in DAGs.* Technical Report 95-52. University of California, Irvine. https://www.ics.uci.edu/~eppstein/pubs/Epp-TR-95-52.pdf

[5] Rudolf Fleischer, Qi Ge, Jian Li, and Hong Zhu. 2007. Efficient Algorithms for $k$-Disjoint Paths Problems on DAGs. In *Algorithmic Aspects in Information and Management (AAIM 2007) (Lecture Notes in Computer Science, Vol. 4508)*. Springer, Berlin, Heidelberg, 134–143. doi:10.1007/978-3-540-72870-2_13

[6] François Le Gall and Florent Urrutia. 2018. Improved Rectangular Matrix Multiplication Using Powers of the Coppersmith–Winograd Tensor. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018)*. SIAM, Philadelphia, PA, USA, 1029–1046. doi:10.1137/1.9781611975031.67

[7] Volker Strassen. 1969. Gaussian Elimination Is Not Optimal. *Numer. Math.* 13 (1969), 354–356. doi:10.1007/BF02165411

[8] Leslie G. Valiant. 1975. General Context-Free Recognition in Less than Cubic Time. *J. Comput. System Sci.* 10, 2 (1975), 308–315. doi:10.1016/S0022-0000(75)80046-8

[9] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2023. New Bounds for Matrix Multiplication: From Alpha to Omega. arXiv:2307.07970v2. https://arxiv.org/abs/2307.07970v2